

75.56220161PEC2Solucion.pdf



Jose_Blasco_83



Fundamentos de Computadores



1º Grado en Ingeniería Informática



**Facultad de Informática, Multimedia y Telecomunicación
Universitat Oberta de Catalunya**



MÁSTER EN

Inteligencia Artificial & Data Management

MADRID

Formamos
talento para un futuro
Sostenible

saber más



Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH



75.562 • Fundamentos de Computadores • PEC2 • 2016-17 • Estudios de Informàtica Multimèdia y Telecomunicación



PEC2 - Segunda prueba de evaluación continuada

Presentación

Después de conocer los sistemas de representación de la información, esta PEC se focaliza en los circuitos lógicos combinacionales. Las funciones lógicas nos permiten describir la funcionalidad de un circuito y mediante las puertas lógicas y los bloques combinacionales podemos implementarlas en un circuito. Todo esto, no se podría hacer sin conocer mecanismos de minimización, como los mapas de Karnaugh, que nos permiten reducir las dimensiones del circuito que se debe implementar.

Competencias

- Conocer y saber aplicar el álgebra de Boole para la manipulación de funciones lógicas.
- Tener nociones tecnológicas de los circuitos digitales y entender la relación entre los circuitos digitales y las funciones lógicas.
- Conocer y saber utilizar las puertas lógicas y los módulos combinacionales en el diseño de circuitos lógicos.

Objetivos

- Saber aplicar las operaciones básicas y los axiomas del álgebra de Boole.
- Saber representar las funciones lógicas mediante Tablas de verdad.
- Saber representar las funciones lógicas mediante expresiones algebraicas.
- Saber analizar un circuito combinacional.
- Saber realizar un cronograma a partir de un circuito digital combinacional.
- Saber sintetizar una función de dos niveles.
- Saber diseñar un circuito combinacional sencillo a partir de los bloques combinacionales explicados en los materiales de la asignatura.

Recursos

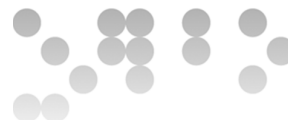
Los recursos que se recomienda usar por esta PEC son los siguientes:

Básicos: El módulo 3 de los materiales.

Complementarios: KeMap, VerilCIRC, VerilChart y el Wiki de la asignatura. También tenéis disponible el servicio PyPAC para obtener beneficios en la asignatura por la realización de ejercicios en VerilUOC.

Criterios de valoración

- Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.



- La valoración está indicada en cada uno de los sub-apartados.

Formato y fecha de entrega

- Para dudas y aclaraciones sobre el enunciado, dirigíos al consultor responsable de vuestra aula.
- Hay que entregar la solución en un fichero PDF usando una de las plantillas entregadas conjuntamente con este enunciado.
- Se tiene que entregar a través de la aplicación de **Registro de EC** de la apartado Evaluación de vuestra aula.
- La fecha límite de entrega es el **4 de noviembre** (a las 24 horas).

Solución de la PEC

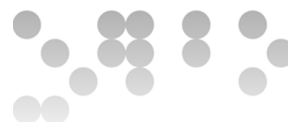
Ejercicio 1 [15%]

Dada la tabla de verdad siguiente:

a	b	c	d	f1	f2
0	0	0	0	0	0
0	0	0	1	1	0
0	0	1	0	1	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	1	0	1
0	1	1	0	0	0
0	1	1	1	0	1
1	0	0	0	0	0
1	0	0	1	0	0
1	0	1	0	0	0
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	0	1
1	1	1	0	0	1
1	1	1	1	1	x

- a) [5%] Expresad sin simplificar la función $f1$ en forma de suma de minterminos.

Los minterminos de la función $f1$ son los productos únicos que se corresponden a las entradas de la tabla de verdad donde la función tiene valor 1. Para construir cada producto sólo hace falta que las variables de entrada que valen 0 estén negadas y las que valen 1 estén sin negar. Una vez identificados y expresados los productos sólo hay que sumarlos para obtener la forma canónica de la función como suma de minterminos.



$$f1(a,b,c,d) = a'b'c'd + a'b'cd + a'b'cd + ab'cd + abcd$$

- b) [10%] Sintetizad de manera mínima en dos niveles la función $f2$ mediante el método de Karnaugh, e implementad el resultado con puertas lógicas.

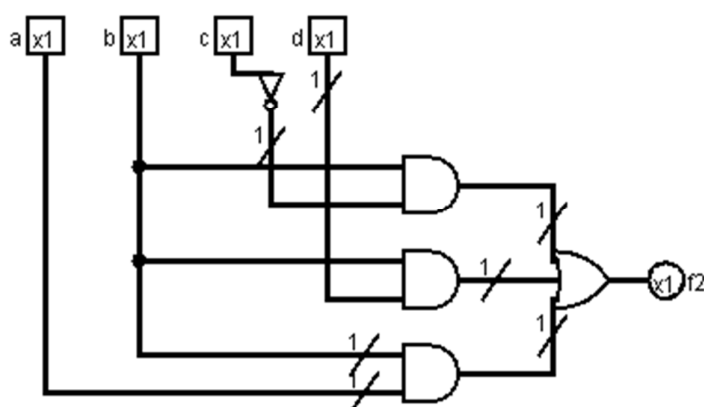
NOTA: Tenéis disponible el ejercicio en VerilCIRC y en KeMAP para verificarlo. En cuanto al mapa de Karnaugh, para evitar un mal uso de KeMAP, en caso de que se hagan más de 5 intentos, la nota del ejercicio se reducirá al 50%. Os recomendamos hacer pruebas con KeMAP con otros ejercicios antes de hacer los de la PEC. En el caso de VerilCIRC para los circuitos, no hay limitación en el número de intentos.

Pasamos los valores de la tabla de verdad al mapa de Karnaugh y agrupamos los unos adyacentes. El mapa de Karnaugh para la función de salida f es el siguiente:

ab \ cd	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	X	0
10	0	0	1	0

$$f2(a,b,c,d) = bc' + bd + ab$$

Dibujamos el circuito correspondiente a la función $f2$:



Importante

Puedo eliminar la publi de este documento con 1 coin

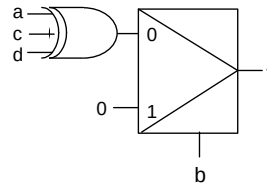
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

75.562 • Fundamentos de Computadores • PEC2 • 2016-17 • Estudios de Informàtica Multimèdia y Telecomunicación



Ejercicio 2 [5%]

Dado el siguiente circuito:

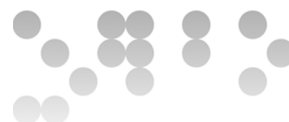


Rellenad la siguiente tabla de verdad:

a	b	c	d	f
0	0	0	0	
0	0	0	1	
0	0	1	0	
0	0	1	1	
0	1	0	0	
0	1	0	1	
0	1	1	0	
0	1	1	1	
1	0	0	0	
1	0	0	1	
1	0	1	0	
1	0	1	1	
1	1	0	0	
1	1	0	1	
1	1	1	0	
1	1	1	1	

NOTA: Tenéis disponible el ejercicio a VerilChart. En este caso no hay limitación en el número de intentos.

a	b	c	d	f
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0

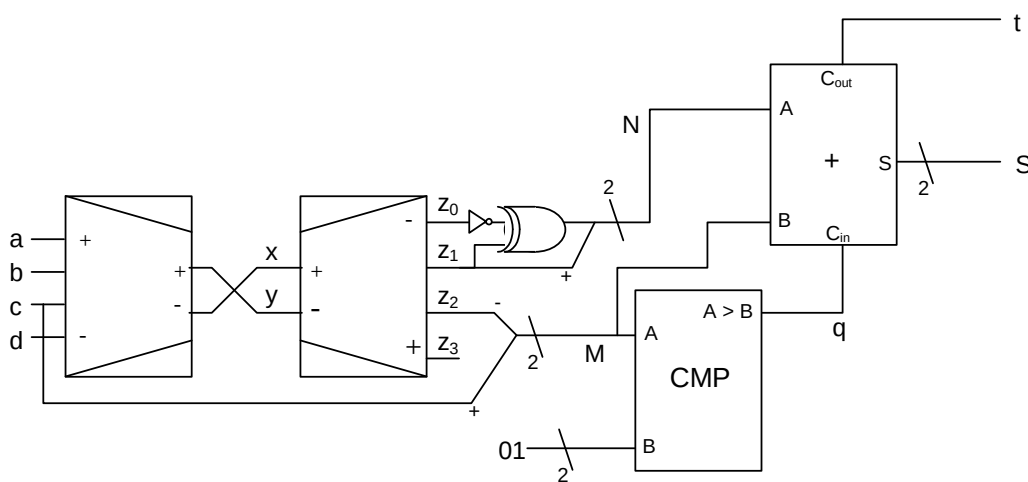


1	1	1	0	0
1	1	1	1	0

Para rellenar la tabla de verdad analizamos el circuito anterior. Para los casos que la entrada b tiene valor 1, la salida f siempre tendrá valor 0. De lo contrario, valdrá el resultado de la función XOR entre las entradas a , c y d .

Ejercicio 3 [20%]

Dado el siguiente circuito lógico combinacional:



Rellenad la siguiente tabla de verdad:

[illegible]

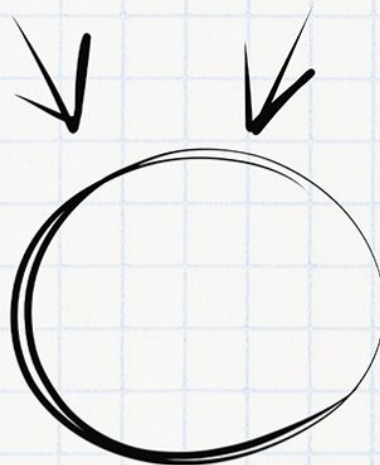
NOTA: Tenéis disponible el ejercicio a VerilChart .

Imagínate aprobando el examen

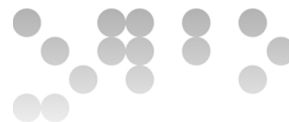
Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH



Las salidas del codificador valen 00 si ninguna variable o sólo la variable a vale 1, 01 si $b = 1$ y $c = d = 0$, 10 si $c = 1$ y $d = 0$, y 11 si $d = 1$. Las señales x e y corresponden a estas salidas en orden inverso.

La señal z_i ($i = 0..3$) vale 1 si las señales $[x, y]$ codifican en binario el número i .

El valor del resto de señales se obtiene de manera muy directa analizando el circuito.

a	b	c	d	x	y	z_3	z_2	z_1	z_0	N	M	q	S	t
0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
0	0	0	1	0	0	0	0	0	1	0	0	0	0	0
0	0	1	0	1	0	0	1	0	0	0	1	1	1	1
0	0	1	1	1	0	0	1	0	0	0	1	1	1	1
0	1	0	0	0	1	0	0	1	0	1	0	0	1	0
0	1	0	1	0	1	0	0	1	0	1	0	0	1	0
0	1	1	0	0	1	0	0	1	0	1	0	1	0	1
0	1	1	1	0	1	0	0	1	0	1	0	1	0	1
1	0	0	0	1	1	1	0	0	0	0	1	0	0	1
1	0	0	1	1	1	1	0	0	0	0	1	0	0	1
1	0	1	0	1	1	1	0	0	0	0	1	1	0	1
1	0	1	1	1	1	1	0	0	0	0	1	1	0	1
1	1	0	0	1	1	1	0	0	0	0	1	0	0	1
1	1	0	1	1	1	1	0	0	0	0	1	0	0	1
1	1	1	0	1	1	1	0	0	0	0	1	1	0	1
1	1	1	1	1	1	1	0	0	0	0	1	1	0	1

Ejercicio 4 [15%]

Diseñad **a nivel de bloques** un circuito con una entrada E de 3 bits, que dé como salida S el número de bits de la entrada que valen 1, codificado en base 2. ¿Cuántos bits debe tener S ? ¿Por qué?

A *nivel de bloques* quiere decir que no se admite construir el circuito partiendo de una tabla de verdad y haciendo la implementación con dos niveles de puertas, sino que se tiene que construir usando bloques combinacionales, y las puertas lógicas que hagan falta. El único bloque combinacional que no está permitido en este ejercicio es la memoria ROM.

NOTA: Tenéis disponible el ejercicio a VerilCIRC.

Primero analizamos el número de bits que debe tener la salida S . Como el máximo número de 1s que pueden haber a la entrada E es 3 (11 en binario), el número de bits de la salida S es 2.

Para implementar este circuito se puede usar un módulo sumador de un bit. Los tres bits de la entrada E se conectan a las entradas del sumador (A , B y Cin) y la salida de

Importante

Puedo eliminar la publi de este documento con 1 coin

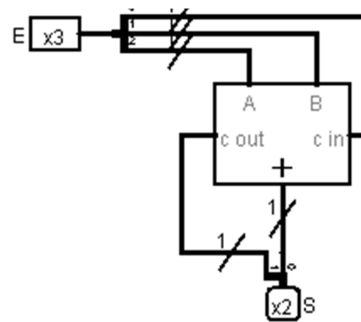
¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

75.562 • Fundamentos de Computadores • PEC2 • 2016-17 • Estudios de Informàtica Multimèdia y Telecomunicación

perdo espacio

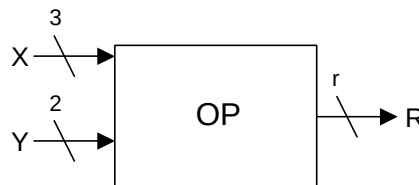


dos bits se construye con la salida del sumador como bit menos significativo y la señal *Cout* como bit más significativo.



Ejercicio 5 [35%]

Se quiere implementar el circuito lógico combinacional *OP*, en el cual todas las señales son números enteros codificados en complemento a 2. La salida *R* tiene que valer $|X \cdot Y|$.



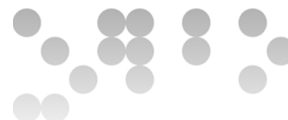
- a) [5%] Si se quiere evitar que en los cálculos se produzca desbordamiento, ¿cuánto debe valer *r*?

En la entrada *X*, con 3 bits y en representación en Ca2, la magnitud más grande que podemos tener en valor absoluto es 4 ($100_{Ca2} = -4$). Por otra parte, en la entrada *Y*, con 2 bits, la magnitud más grande es 2 ($10_{Ca2} = -2$). En consecuencia, el valor máximo que podemos obtener de la multiplicación es 8 que para ser representado en Ca2 necesita **5 bits** con el cual se puede representar el rango $[-16, 15]$.

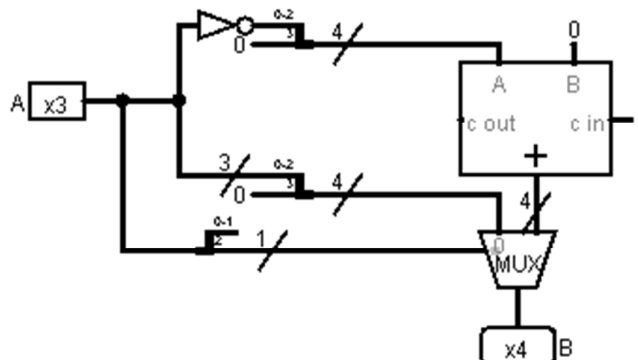
- b) [10%] Implementad a nivel de bloques un bloque combinacional *ABS* que calcule el valor absoluto de una señal de entrada *A*, representada en Ca2 con 3 bits. Justificad el número de bits de la salida del circuito, llamada *B*.

Cómo *A* es un número en Ca2 de 3 bits, estará en el rango $[-4, +3]$, en consecuencia, para representar el valor absoluto $|A|$ necesitaremos hasta **4 bits** para representar el valor positivo 4 (0100_{Ca2}).

En el circuito siguiente controlamos los valores positivos y negativos a partir de un multiplexor controlado por el bit más significativo de la entrada *A*. En el caso de un valor positivo, la salida será el mismo número con la adición de un 0 como bit más significativo. De lo contrario, en caso de valor negativo, cambiamos de signo el número. Para hacerlo, en una representación en Ca2,

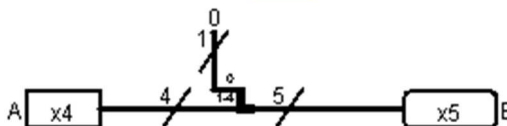


complementamos todos los bits y sumamos 1. Notad que después de complementar los bits del número, hemos añadido también un 0 como bit más significativo para lograr la dimensión de 4 bits.



- c) [5%] Implementad **a nivel de bloques** un bloque combinacional *DOUBLE* que multiplique por 2 una señal de entrada A, representada en Ca2 con 4 bits. Justificad el número de bits de la salida del circuito, llamada B.

Para multiplicar por 2 un valor en Ca2 sólo hay que desplazar los bits del número una posición a la izquierda y añadir un 0 a la derecha. En consecuencia, para poder representar la salida de la operación en cualquier número de 4 bits en Ca2 necesitamos **5 bits**.

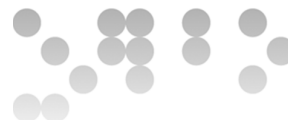


- d) [15%] Implementad **a nivel de bloques** el circuito *OP*. Podéis usar los bloques obtenidos en los apartados anteriores y adaptarlos, si hace falta, a cualquier número de bits de entrada. También podéis utilizar cualquiera otro bloque combinacional, excepto una memoria ROM. Indicad la anchura de todos los buses del circuito.

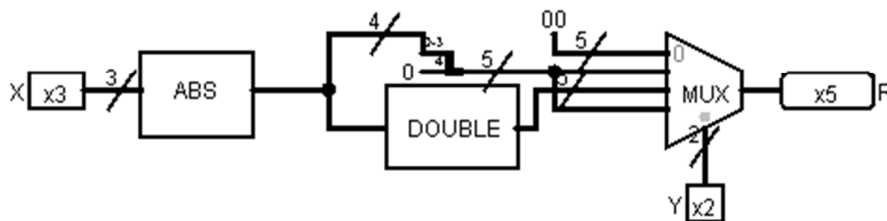
NOTA: Tenéis disponible los apartados b), c) y d) a VerilCIRC. En el caso del apartado d), VerilUOC no acepta crear subcircuitos. Por lo tanto, para utilizar los bloques de los apartados anteriores tendréis que copiar todos los bloques que habéis utilizado para diseñarlos al nuevo circuito.

Este circuito usa un módulo *ABS* del apartado b) para calcular el valor absoluto de la entrada X y un multiplexor para tratar los diferentes casos de la entrada Y.

- Si Y es 0, la multiplicación da 0 (entrada menos significativa del multiplexor),
- si Y es 1 o -1 (01_{Ca2} o 11_{Ca2}), la multiplicación da $|X|$ que ya tenemos calculado y al que sólo hay que añadir un 0 para llegar a los 5 bits necesarios.
- Finalmente, si Y vale -2 (10_{Ca2}) hay que doblar $|X|$, lo cual hacemos con



el bloque *DOUBLE* de la apartado c).



Nota importante: Todo el mundo debe responder a las dos preguntas siguientes. Según si participáis o no en la prueba piloto del proyecto TeSLA la forma de entregarlas es diferente:

1. Todo alumno que **participe** en el piloto debe ir al Moodle (Moodle Tesla) al que se puede acceder desde el aula y responder a las preguntas en este espacio. Encontraréis las instrucciones dentro del Moodle. **La activación de las preguntas se hará el 2 de noviembre.**
2. Todo alumno que **NO participe** en el piloto debe responder a las preguntas dentro del documento de entrega de la PEC.

Ejercicio 6 [5%] para responder de forma escrita utilizando Keystroke dynamics

Describid como implementaríais un bloque codificador a nivel de puertas. Dad la respuesta en cinco líneas como máximo.

Sabemos que un módulo codificador tiene m salidas y 2^m entradas, cada una de las m salidas se puede ver como una función de las 2^m entradas. Así que podemos hacer la tabla de verdad correspondiente, encontrar los minterminos e, incluso, aplicar algún método de optimización. De este modo podemos construir un circuito de dos niveles de puertas para cada salida.

Ejercicio 7 [5%] para responder mediante una grabación de vídeo y voz

Describid como implementaríais un cierto número de funciones mediante una memoria ROM. ¿Cómo se determina la medida de la ROM?

Conectamos las variables de las funciones a la entrada de direcciones de la memoria ROM (así, tendremos 2^n palabras de memoria donde n es el número de variables de entrada). En cada palabra de memoria almacenamos los valores de salida de cada función por la combinación de entradas correspondiente (así, cada palabra tendrá tantos bits como funciones queramos implementar).

Finalmente, la medida en bits de la ROM será $2^n \cdot$ número de funciones a implementar.